PACKAGES AND EXCEPTIONS

Packages: Packages, Packages and Member Access, Importing Packages.
Exceptions: Exception-Handling Fundamentals, Exception Types, Uncaught Exceptions, Using try and catch, Multiple catch Clauses, Nested try Statements, throw, throws, finally, Java's Built-in Exceptions, Creating Your Own Exception Subclasses, Chained Exceptions.

Object class in Java

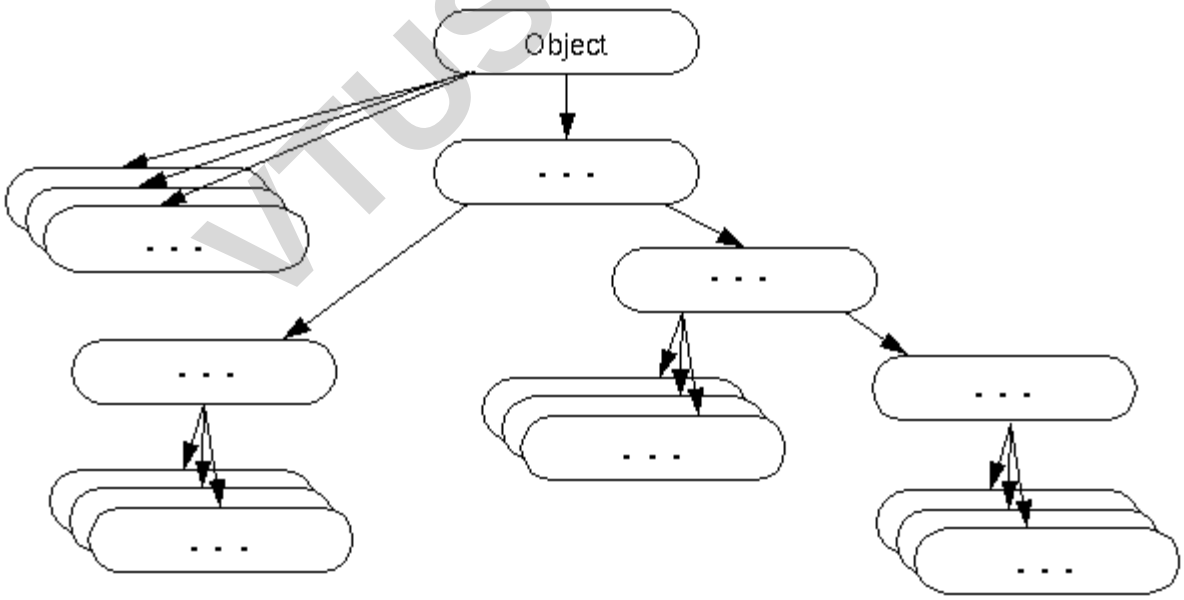
The **Object class** is the parent class of all the classes in java by default. In other words, it is the topmost class of java.

The Object class is beneficial if you want to refer any object whose type you don't know. Notice that parent class reference variable can refer the child class object, know as upcasting.

Let's take an example, there is getObject() method that returns an object but it can be of any type like Employee,Student etc, we can use Object class reference to refer that object. For example:

1. Object obj=getObject();//we don't know what object will be returned from this method

The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc.



Methods of Object class

The Object class provides many methods. They are as follows:

Method	Description
public final Class getClass()	returns the Class object of this object. The Class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.
public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

Java Package

1. **Java Package**
2. **Example of package**
3. **Accessing package**
    1. **By import packagename.***
    2. **By import packagename.classname**
    3. **By fully qualified name**
    4. **Subpackage**
    5. **Sending class file to another directory**
    6. **-classpath switch**
    7. **4 ways to load the class file or jar file**
    8. **How to put two public class in a package**
    9. **Static Import**
    10. **Package class**

A **java package** is a group of similar types of classes, interfaces and sub-packages.

Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

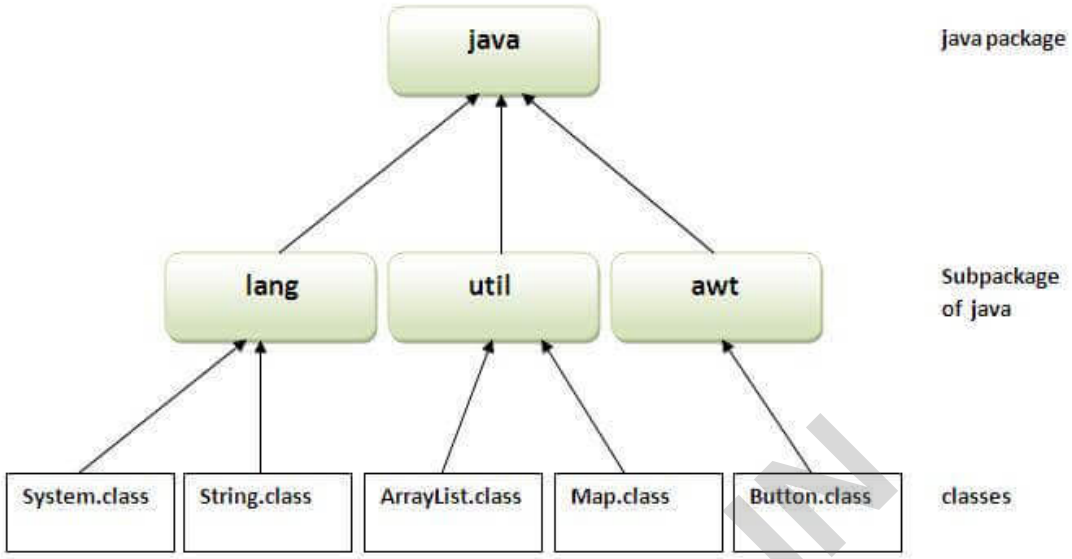
Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.

2) Java package provides access protection.

3) Java package removes naming collision.



Simple example of java package

The **package keyword** is used to create a package in java.

```
//save as Simple.java
package mypack;
public class Simple
{
    public static void main(String args[])
    {
      System.out.println("Welcome to package");
     }
}
```

How to compile java package

If you are not using any IDE, you need to follow the **syntax** given below:

1. javac -d directory javafilename

For **example**

1. javac -d . Simple.java

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

To Compile: javac -d . Simple.java
To Run: java mypack.Simple

Output:Welcome to package

The -d is a switch that tells the compiler where to put the class file i.e. it represents destination. The . represents the current folder.

How to access package from another package?

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

1) Using packagename.*

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.

The import keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

//save by A.java

```
package pack;
public class A
{
      public void msg()
      {
         System.out.println("Hello");
     }
}
//save by B.java
package mypack;
import pack.*;
 class B
 {
       public static void main(String args[])
      {
       A obj = new A();
       obj.msg();
      }
}
```

Output:Hello

2) Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

Example of package by import package.classname

```
//save by A.java

package pack;
public class A
{
      public void msg()
      {
```

```
        System.out.println("Hello");
    }
}
//save by B.java
package mypack;
import pack.A;
 class B
 {
        public static void main(String args[])
        {
         A obj = new A();
         obj.msg();
        }
}
```

Output:Hello

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

```
//save by A.java
package pack;
public class A
{
     public void msg()
     {
       System.out.println("Hello");
     }
}
//save by B.java
```

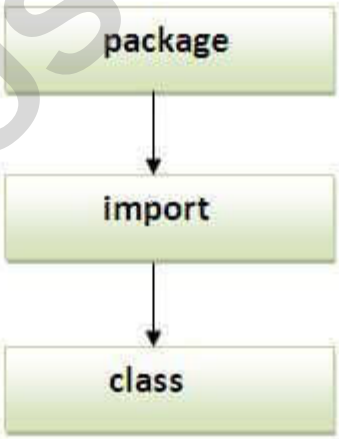
```
package mypack;
class B
{
 public static void main(String args[])
 {
  pack.A obj = new pack.A();//using fully qualified name
  obj.msg();
 }
}
```

Output:Hello

Note: If you import a package, subpackages will not be imported.

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Note: Sequence of the program must be package then import then class.



Subpackage in java

Package inside the package is called the **subpackage**. It should be created **to categorize the package further**.

Let's take an example, Sun Microsystem has definded a package named java that contains many classes like System, String, Reader, Writer, Socket etc. These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on. So, Sun has

subcategorized the java package into subpackages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

The standard of defining package is domain.company.package e.g. com.javatpoint.bean or org.sssit.dao.

Example of Subpackage

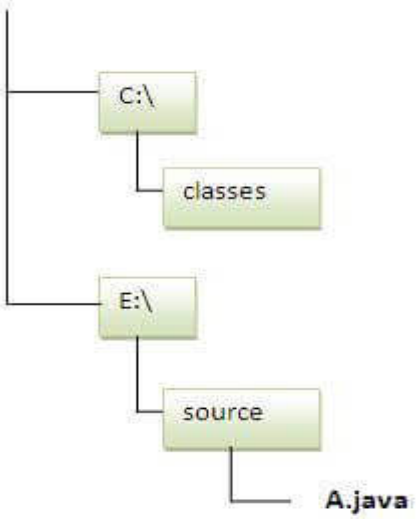
```
package com.javatpoint.core;
class Simple
{
      public static void main(String args[])
     {
      System.out.println("Hello subpackage");
      }
}
```

To Compile: javac -d . Simple.java
To Run: java com.javatpoint.core.Simple

Output:Hello subpackage

How to send the class file to another directory or drive?

There is a scenario, I want to put the class file of A.java source file in classes folder of c: drive. For example:




```
//save as Simple.java

package mypack;

public class Simple

{

     public static void main(String args[])

    {

       System.out.println("Welcome to package");

      }

}
```

To Compile:

e:\sources> javac -d c:\classes Simple.java

To Run:

To run this program from e:\source directory, you need to set classpath of the directory where the class file resides.

e:\sources> set classpath=c:\classes;.;

e:\sources> java mypack.Simple

Another way to run this program by -classpath switch of java:

The -classpath switch can be used with javac and java tool.

To run this program from e:\source directory, you can use -classpath switch of java that tells where to look for class file. For example:

e:\sources> java -classpath c:\classes mypack.Simple

Output:Welcome to package

Ways to load the class files or jar files

There are two ways to load the class files temporary and permanent.

- **Temporary**
  - By setting the classpath in the command prompt
  - By -classpath switch
- **Permanent**
  - By setting the classpath in the environment variables
  - By creating the jar file, that contains all the class files, and copying the jar file in the jre/lib/ext folder.

Rule: There can be only one public class in a java source file and it must be saved by the public class name.

```
//save as C.java otherwise Compilte Time Error

class A
{
}
class B
```

```
{
}
public class C
{
}
```

How to put two public classes in a package?

If you want to put two public classes in a package, have two java source files containing one public class, but keep the package name same. For example:

```
//save as A.java

package javatpoint;
public class A
{
}
//save as B.java

package javatpoint;
public class B
{
}
```


Access Modifiers in Java

1. **Private access modifier**
2. **Role of private constructor**
3. **Default access modifier**
4. **Protected access modifier**
5. **Public access modifier**
6. **Access Modifier with Method Overriding**

There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are **four types** of Java access modifiers:

1. **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

Understanding Java Access Modifiers

Let's understand the access modifiers in Java by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

1) Private

The private access modifier is accessible only within the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```
class A
{
private int data=40;
    private void msg()
    {
    System.out.println("Hello java");
    }
}

public class Simple
{
     public static void main(String args[])
    {
      A obj=new A();
      System.out.println(obj.data);//Compile Time Error
      obj.msg();//Compile Time Error
      }
}
```


Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

```
class A
{
    private A()
    {
    }//private constructor
    void msg()
```

```
    {
    System.out.println("Hello java");
    }
}
public class Simple
{
 public static void main(String args[])
{
  A obj=new A();//Compile Time Error
 }
}
```

Note: A class cannot be private or protected except nested class.

2) Default

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```
//save by A.java
package pack;
class A
{
    void msg()
```

```
    {
    System.out.println("Hello");
    }
}
//save by B.java
package mypack;
import pack.*;
class B
{
        public static void main(String args[])
       {
        A obj = new A();//Compile Time Error
        obj.msg();//Compile Time Error
        }
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) Protected

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifer.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A
{
    protected void msg()
    {
    System.out.println("Hello");
    }
}
//save by B.java
package mypack;
import pack.*;

class B extends A
{
   public static void main(String args[])
  {
    B obj = new B();
    obj.msg();
   }
}
```

Output:Hello

4) Public

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
//save by A.java

package pack;
public class A
{
    public void msg()
    {
    System.out.println("Hello");
    }
}
//save by B.java

package mypack;
import pack.*;

class B
{
  public static void main(String args[])
{
   A obj = new A();
   obj.msg();
  }
}
```

Output:Hello

Java Access Modifiers with Method Overriding

If you are overriding any method, overridden method (i.e. declared in subclass) must not be more restrictive.

```
class A
{
        protected void msg()
        {
        System.out.println("Hello java");
        }
}
public class Simple extends A
{
     void msg()
     {
     System.out.println("Hello java"); //C.T.Error
     }
      public static void main(String args[])
     {
       Simple obj=new Simple();
       obj.msg();
       }
}
```


The default modifier is more restrictive than protected. That is why, there is a compile-time error.

Exception Handling in Java

1. **Exception Handling**
2. **Advantage of Exception Handling**
3. **Hierarchy of Exception classes**
4. **Types of Exception**
5. **Exception Example**
6. **Scenarios where an exception may occur**

The **Exception Handling in Java** is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.

What is Exception in Java?

Dictionary Meaning: Exception is an abnormal condition.

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

What is Exception Handling?

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IO Exception, SQL Exception, RemoteException, etc.

Advantage of Exception Handling

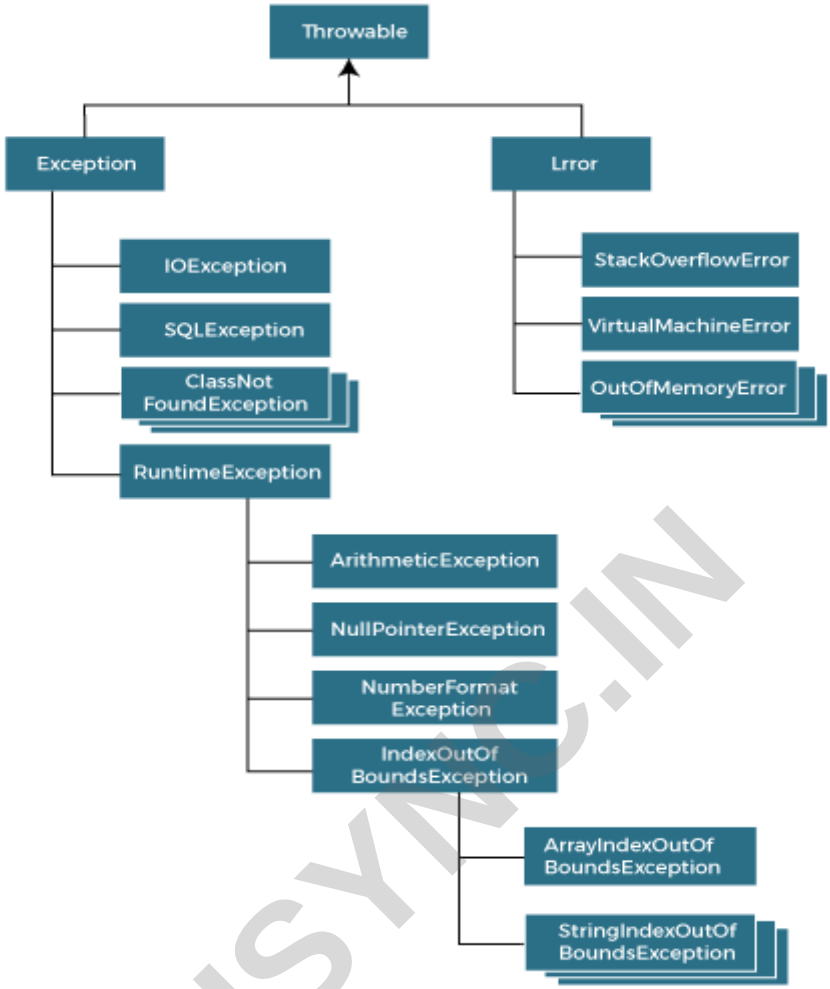
The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

```
statement 1;

statement 2;

statement 3;  ;//exception occurs

statement 4;

statement 5
```

Suppose there are 5 statements in a Java program and an exception occurs at statement 3; the rest of the code will not be executed, i.e., statements 4 to 5 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Hierarchy of Java Exception classes

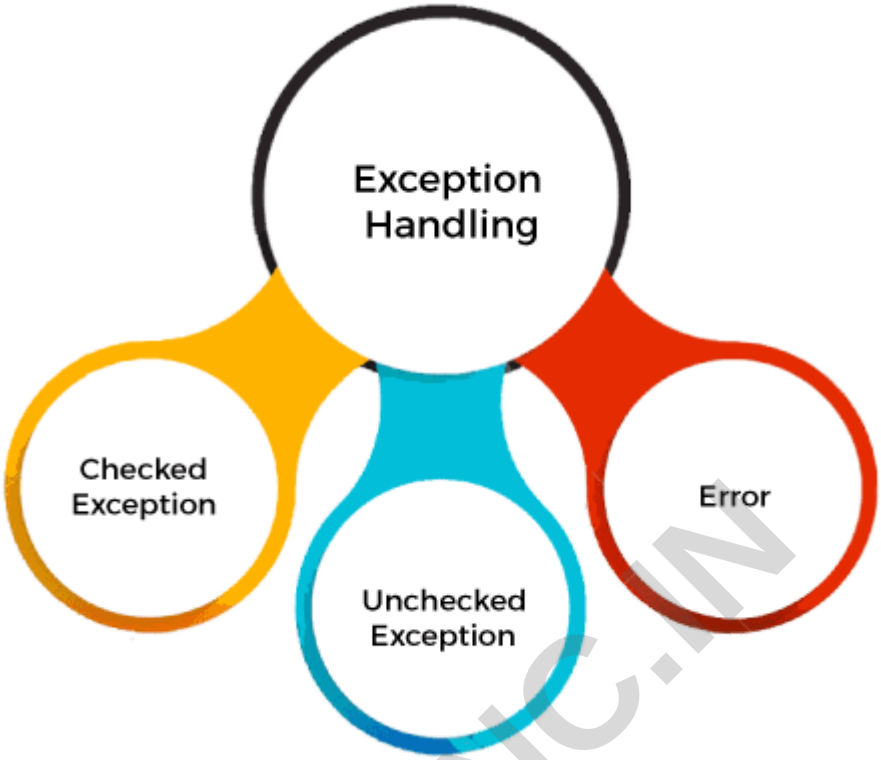
The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. **Checked Exception**
2. **Unchecked Exception**
3. **Error(unchecked Exception)**



Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except Runtime Exception and Error are known as checked exceptions.

For example,

IO Exception,

SQL Exception, etc.

Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the Runtime Exception are known as unchecked exceptions.

For example,

Arithmetic Exception,

NullPointerException,

ArrayIndexOutOfBoundsException, etc.

Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable.

Some examples of errors are

OutOfMemoryError,

VirtualMachineError,

Assertion Error etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

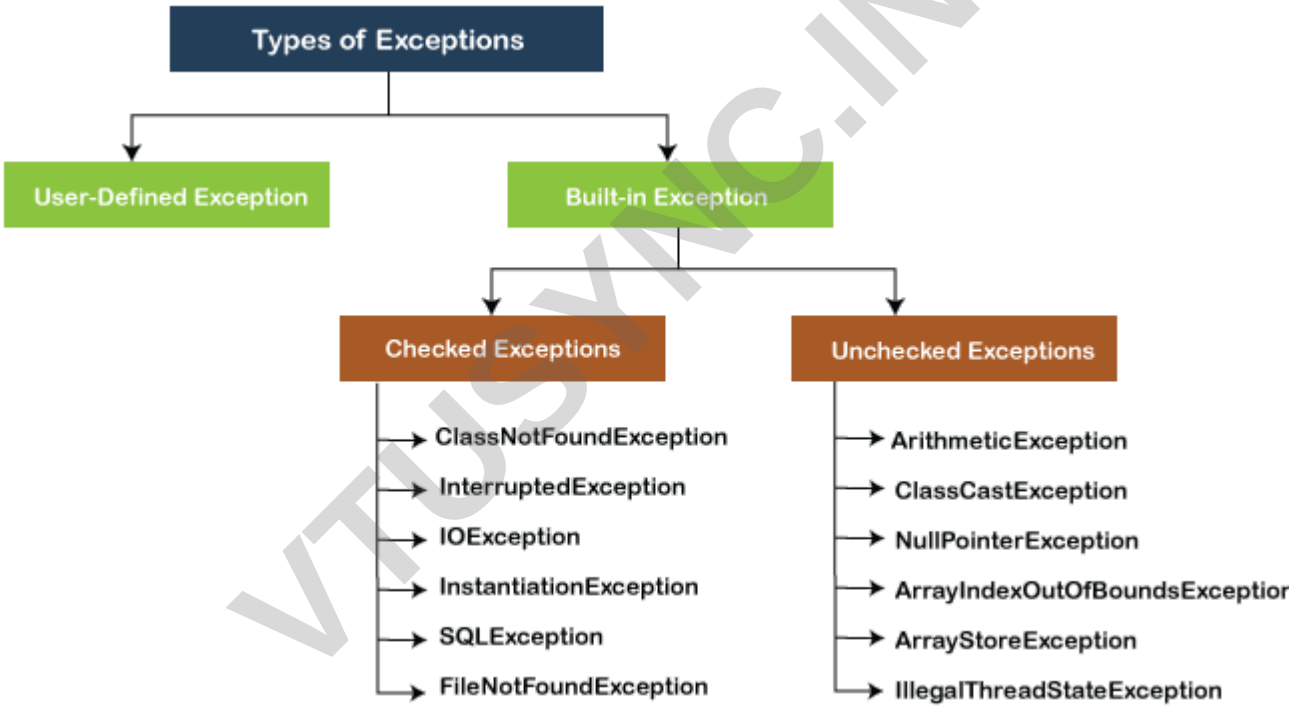
Keyword	Description
Try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
Catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
Finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
Throw	The "throw" keyword is used to throw an exception.
Throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Types of Exception in Java

In Java, **exception** is an event that occurs during the execution of a program and disrupts the normal flow of the program's instructions. Bugs or errors that we don't want and restrict our program's normal execution of code are referred to as **exceptions**. In this section, we will focus on the **types of exceptions in Java** and the differences between the two.

Exceptions can be categorized into two ways:

1. **Built-in Exceptions**
   - Checked Exception
   - Unchecked Exception
2. **User-Defined Exceptions**



Built-in Exception

Exceptions that are already available in **Java libraries** are referred to as **built-in exception**. These exceptions are able to define the error situation so that we can understand the reason of getting this error. It can be categorized into two broad categories, i.e., **checked exceptions** and **unchecked exception**.

Checked Exception

Checked exceptions are called **compile-time** exceptions because these exceptions are checked at compile-time by the compiler. The compiler ensures whether the programmer handles the exception or not. The programmer should have to handle the exception; otherwise, the system has shown a compilation error.

CheckedExceptionExample.java

```
import java.io.*;
class CheckedExceptionExample
 {
 public static void main(String args[])
 {
  FileInputStream file_data = null;
  file_data = new FileInputStream("C:/Users/ajeet/OneDrive/Desktop/Hello.txt");
  int m;
  while(( m = file_data.read() ) != -1)
 {
     System.out.print((char)m);
  }
  file_data.close();
}
 }
```


In the above code, we are trying to read the **Hello.txt** file and display its data or content on the screen. The program throws the following exceptions:

1. The **FileInputStream(File filename)** constructor throws the **FileNotFoundException** that is checked exception.
2. The **read()** method of the **FileInputStream** class throws the **IOException**.
3. The **close()** method also throws the IOException.

Output:

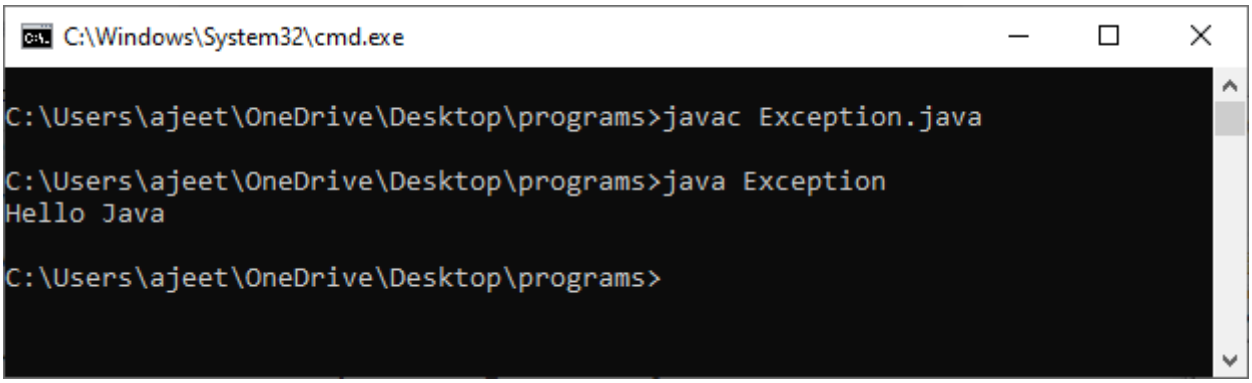
How to resolve the error?

There are basically two ways through which we can solve these errors.

1) The exceptions occur in the main method. We can get rid from these compilation errors by declaring the exception in the main method using the **throws** We only declare the IOException, not FileNotFoundException, because of the child-parent relationship. The IOException class is the parent class of FileNotFoundException, so this exception will automatically cover by IOException. We will declare the exception in the following way:

```
class Exception
{
   public static void main(String args[])  throws IOException
    {
      ...
      ...
     }
}
```

If we compile and run the code, the errors will disappear, and we will see the data of the file.



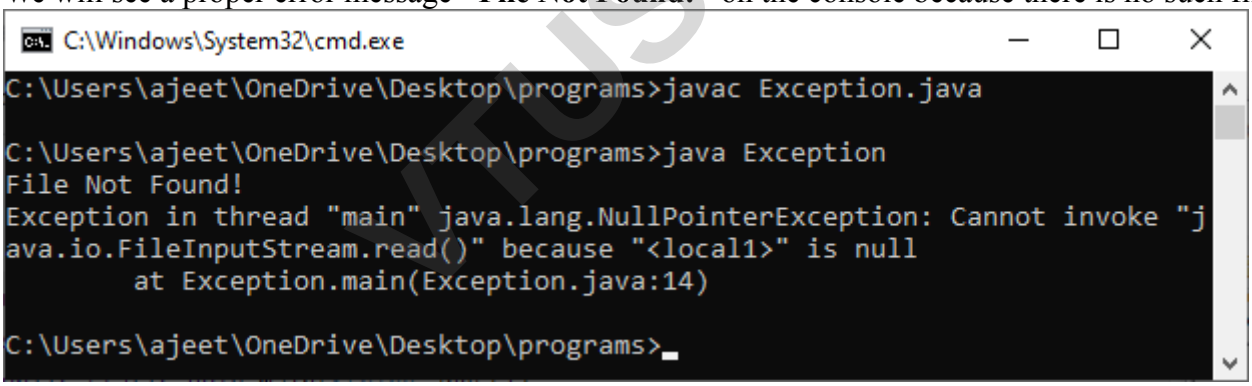
2) We can also handle these exception using **try-catch** However, the way which we have used above is not correct. We have to a give meaningful message for each exception type. By doing that it would be easy to understand the error. We will use the try-catch block in the following way:

Exception.java

```
import java.io.*;
class Exception
{
   public static void main(String args[])
   {
      FileInputStream file_data = null;
      try
      {
file_data = new FileInputStream("C:/Users/ajeet/OneDrive/Desktop/programs/Hell.txt");      }
catch(FileNotFoundException fnfe)
{
        System.out.println("File Not Found!");
 }

 int m;
```

```
        try
        {
           while(( m = file_data.read() ) != -1)
            {
              System.out.print((char)m);
            }
        file_data.close();
        }
      catch(IOException ioe)
       {
           System.out.println("I/O error occurred: "+ioe);
        }
     }
}
```

We will see a proper error message **"File Not Found!"** on the console because there is no such file in that location.



Unchecked Exceptions

The **unchecked** exceptions are just opposite to the **checked** exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error. Usually, it occurs when the user provides bad data during the interaction with the program.

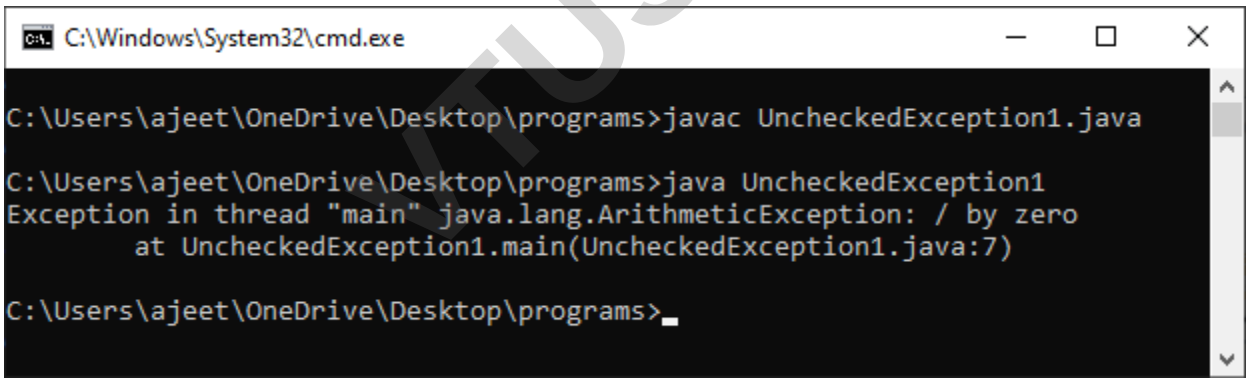
Note: The RuntimeException class is able to resolve all the unchecked exceptions because of the child-parent relationship.

UncheckedExceptionExample1.java

```
class UncheckedExceptionExample1
{
  public static void main(String args[])
  {
   int a = 35;
   int zero = 0;
   int result = a/zero;
   //Give Unchecked Exception here.
    System.out.println(result);
  }
}
```

In the above program, we have divided 35 by 0. The code would be compiled successfully, but it will throw an ArithmeticException error at runtime. On dividing a number by 0 throws the divide by zero exception that is a uncheck exception.

Output:

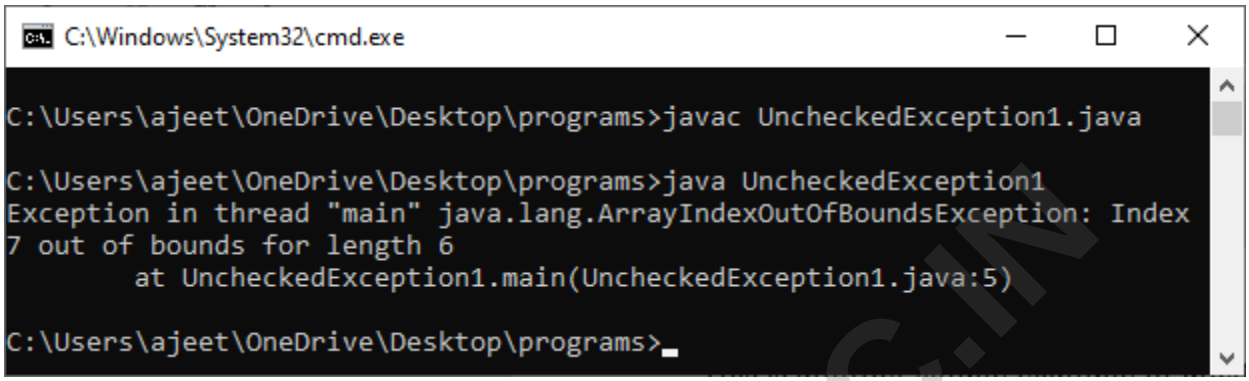


UncheckedException1.java

```
class UncheckedException1
{
  public static void main(String args[])
  {
```

```
        int num[] ={10,20,30,40,50,60};
        System.out.println(num[7]);
      }
    }
```

Output:



In the above code, we are trying to get the element located at position 7, but the length of the array is 6. The code compiles successfully, but throws the ArrayIndexOutOfBoundsException at runtime.

User-defined Exception

In Java, we already have some built-in exception classes like

ArrayIndexOutOfBoundsException

NullPointerException, and **ArithmeticException**.

These exceptions are restricted to trigger on some predefined conditions. In Java, we can write our own exception class by extends the Exception class. We can throw our own exception on a particular condition using the throw keyword. For creating a user-defined exception, we should have basic knowledge of **the** try-catch block and **throw** keyword

.

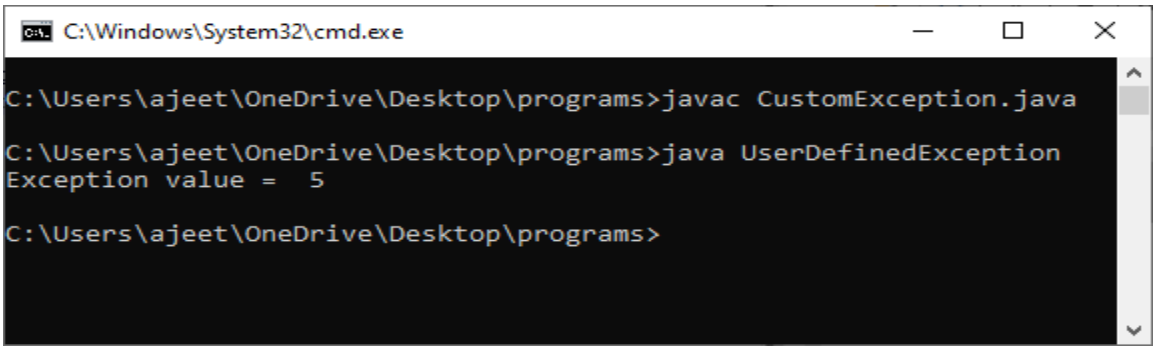
Let's write a Java program and create user-defined exception.

UserDefinedException.java

```
import java.util.*;
class UserDefinedException
```

```
{
   public static void main(String args[])
    {
      try
      {
         throw new NewException(5);
      }
      catch(NewException ex)
      {
         System.out.println(ex) ;
      }
   }
}
class NewException extends UserDefinedException
{
   int x;
   NewException(int y)
   {
      x=y;
    }
   public String toString()
   {
      return ("Exception value =  "+x) ;
   }
}
```

Output:



Description:

In the above code, we have created two classes, i.e., **UserDefinedException** and **NewException**. The **UserDefinedException** has our main method, and the **NewException** class is our user-defined exception class, which extends **exception**. In the **NewException** class, we create a variable **x** of type integer and assign a value to it in the constructor. After assigning a value to that variable, we return the exception message.

In the **UserDefinedException** class, we have added a **try-catch** block. In the try section, we throw the exception, i.e., **NewException** and pass an integer to it. The value will be passed to the NewException class and return a message. We catch that message in the catch block and show it on the screen.

Difference Between Checked and Unchecked Exception

S.No	Checked Exception	Unchecked Exception
1.	These exceptions are checked at compile time. These exceptions are handled at compile time too.	These exceptions are just opposite to the checked exceptions. These exceptions are not checked and handled at compile time.
2.	These exceptions are direct subclasses of Exception but not extended from RuntimeException class.	They are the direct subclasses of the RuntimeException class.
3.	The code gives a compilation error in the case when a method throws a checked exception. The compiler is not able to handle the exception on its own.	The code compiles without any error because the exceptions escape the notice of the compiler. These exceptions are the results of user-created errors in programming logic.
4.	These exceptions mostly occur when the probability of failure is too high.	These exceptions occur mostly due to programming mistakes.
5.	Common checked exceptions include IOException, DataAccessException, InterruptedException, etc.	Common unchecked exceptions include ArithmeticException, InvalidClassException, NullPointerException, etc.

6.	These exceptions are propagated using the throws keyword.	These are automatically propagated.
7.	It is required to provide the try-catch and try-finally block to handle the checked exception.	In the case of unchecked exception it is not mandatory.

Bugs or errors that we don't want and restrict the normal execution of the programs are referred to as **exceptions**.

ArithmeticException,
ArrayIndexOutOfBoundExceptions,
ClassNotFoundExceptions etc. are come in the category of **Built-in Exception**. Sometimes, the built-in exceptions are not sufficient to explain or describe certain situations. For describing these situations, we have to create our own exceptions by creating an exception class as a subclass of the **Exception** class. These types of exceptions come in the category of **User-Defined Exception**.

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```
public class JavaExceptionExample
{
 public static void main(String args[])
 {
  try
 {
    //code that may raise exception
    int data=100/0;
 }
 catch(ArithmeticException e)
 {
    System.out.println(e);
 }
 //rest code of the program
```

```
        System.out.println("rest of the code...");
    }
}
```

Tet Now

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...
```

In the above example, 100/0 raises an ArithmeticException which is handled by a try-catch block.

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
1. int a=50/0;//ArithmeticException
```

2) A scenario where NullPointerException occurs

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

```
1. String s=null;
2. System.out.println(s.length());//NullPointerException
```

3) A scenario where NumberFormatException occurs

If the formatting of any variable or number is mismatched, it may result into NumberFormatException. Suppose we have a string variable that has characters; converting this variable into digit will cause NumberFormatException.

```
1. String s="abc";
2. int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where ArrayIndexOutOfBoundsException occurs

When an array exceeds to it's size, the ArrayIndexOutOfBoundsException occurs. there may be other reasons to occur ArrayIndexOutOfBoundsException. Consider the following statements.

```
1. int a[]=new int[5];
```

2. a[10]=50; //ArrayIndexOutOfBoundsException

Java try-catch block

Java try block

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
try
{
//code that may throw an exception
}
catch(Exception_class_Name ref)
{
}
```

Syntax of try-finally block

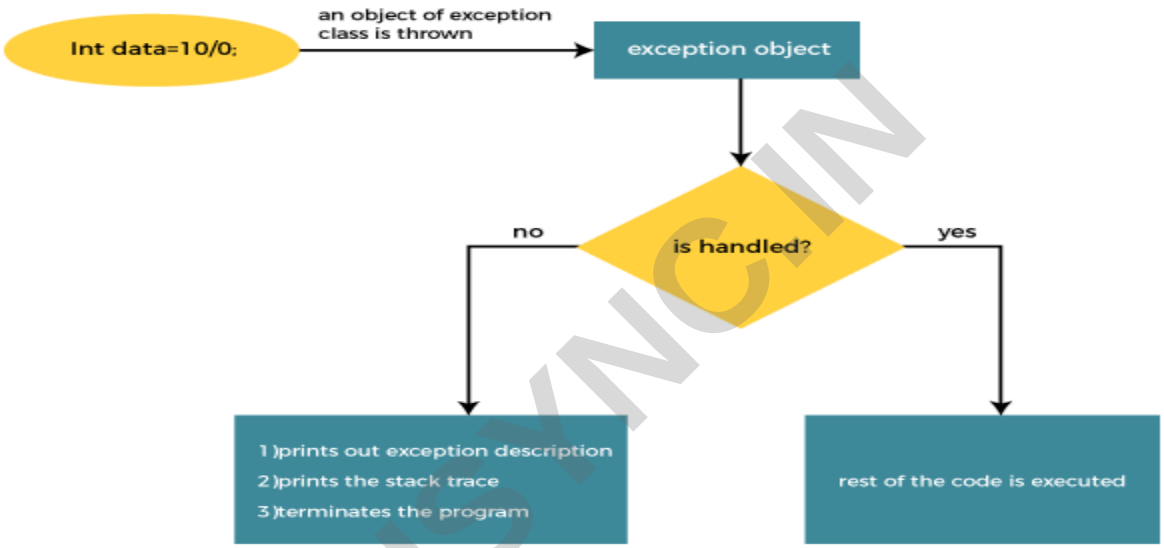
```
try
{
//code that may throw an exception
}
finally
{
}
```

Java catch block

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Internal Working of Java try-catch block



The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

Problem without exception handling

Let's try to understand the problem if we don't use a try-catch block.

Example 1

TryCatchExample1.java

```
public class TryCatchExample1
```

```
{
    public static void main(String[] args)
    {
        int data=50/0; //may throw exception
        System.out.println("rest of the code");
        }
}
```

Test it Now

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

As displayed in the above example, the **rest of the code** is not executed (in such case, the **rest of the code** statement is not printed).

Solution by exception handling

Let's see the solution of the above problem by a java try-catch block.

Example 2

TryCatchExample2.java

```
public class TryCatchExample2
{

  public static void main(String[] args)
  {
    try
    {
    int data=50/0; //may throw exception
    }
      //handling the exception
    catch(ArithmeticException e)
```

```
        {
            System.out.println(e);
        }
        System.out.println("rest of the code");
    }

}
```

Test it Now

Output:

```
java.lang.ArithmeticException: / by zero
rest of the code
```

As displayed in the above example, the **rest of the code** is executed, i.e., the **rest of the code** statement is printed.

Example 3

Let's see an example to print a custom message on exception.

TryCatchExample3.java

```
public class TryCatchExample3
 {

   public static void main(String[] args)
   {
```

```
        try
        {
        int data=50/0; //may throw exception
        }
            // handling the exception
        catch(Exception e)
        {
                // displaying the custom message
           System.out.println("Can't divided by zero");
        }
    }

}
```

Test it Now

Output:

```
Can't divided by zero
```


Java Catch Multiple Exceptions

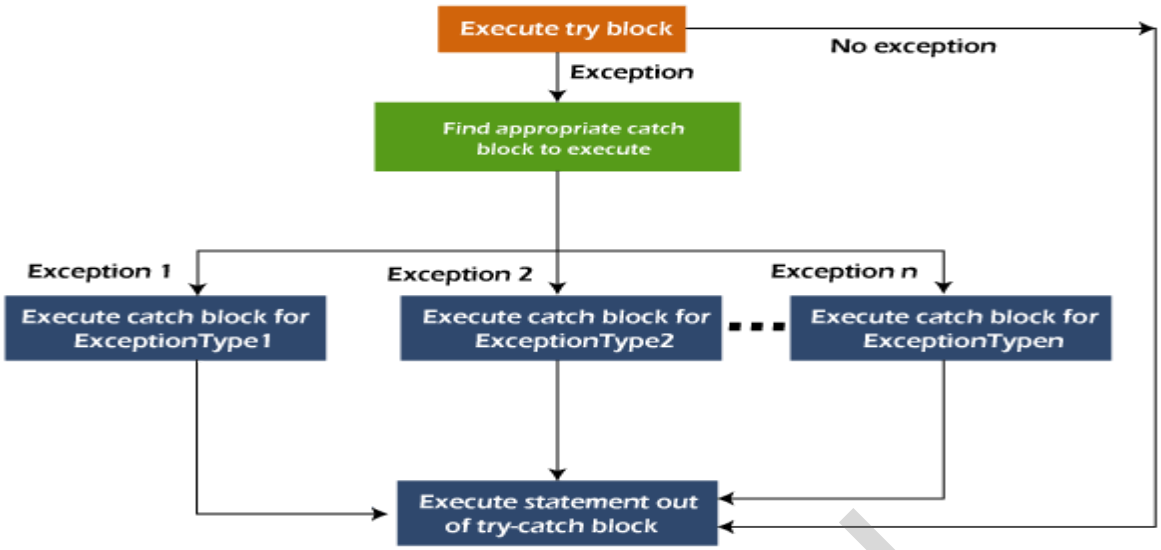
Java Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Flowchart of Multi-catch Block



Example 1

Let's see a simple example of java multi-catch block.

MultipleCatchBlock1.java

```
public class MultipleCatchBlock1
 {
    public static void main(String[] args)
    {
          try
          {
           int a[]=new int[5];
           a[5]=30/0;
          }
          catch(ArithmeticException e)
            {
             System.out.println("Arithmetic Exception occurs");
            }
          catch(ArrayIndexOutOfBoundsException e)
            {
             System.out.println("ArrayIndexOutOfBounds Exception occurs");
            }
```

```
        catch(Exception e)
          {
           System.out.println("Parent Exception occurs");
          }
        System.out.println("rest of the code");
    }}
```

Output:

```
Arithmetic Exception occurs
rest of the code
```

Java Nested try block

In Java, using a try block inside another try block is permitted. It is called as nested try block. Every statement that we enter a statement in try block, context of that exception is pushed onto the stack.

For example,

the **inner try block** can be used to handle **ArrayIndexOutOfBoundsException** while the **outer try block** can handle the **ArithemeticException** (division by zero).

Why use nested try block

Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

Syntax:

```
....
//main try block
try
{
   statement 1;
   statement 2;
//try catch block within another try block
   try
```

```
    {
        statement 3;
        statement 4;
//try catch block within nested try block
        try
        {
            statement 5;
            statement 6;
    }
        catch(Exception e2)
        {
//exception message
        }
      }
    catch(Exception e1)
    {
//exception message
    }
}
//catch block of parent (outer) try block
catch(Exception e3)
{
//exception message
```

```
}
....
```

Java Nested try Example

Example 1

Let's see an example where we place a try block within another try block for two different exceptions.

NestedTryBlock.java

```
public class NestedTryBlock
{
 public static void main(String args[])
{
 //outer try block
  try
  {
 //inner try block 1
   try
  {
    System.out.println("going to divide by 0");
    int b =39/0;
  }
   //catch block of inner try block 1
   catch(ArithmeticException e)
   {
     System.out.println(e);
   }
  //inner try block 2
   try
  {
   int a[]=new int[5];

   //assigning the value out of array bounds
```

```
    a[5]=4;
    }
    //catch block of inner try block 2
   catch(ArrayIndexOutOfBoundsException e)
   {
     System.out.println(e);
   }
   System.out.println("other statement");
  }
  //catch block of outer try block
  catch(Exception e)
  {
   System.out.println("handled the exception (outer catch)");
  }

  System.out.println("normal flow..");
 }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac NestedTryBlock.java

C:\Users\Anurati\Desktop\abcDemo>java NestedTryBlock
going to divide by 0
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
other statement
normal flow..
```

When any try block does not have a catch block for a particular exception, then the catch block of the outer (parent) try block are checked for that exception, and if it matches, the catch block of outer try block is executed.

If none of the catch block specified in the code is unable to handle the exception, then the Java runtime system will handle the exception. Then it displays the system generated message for that exception.

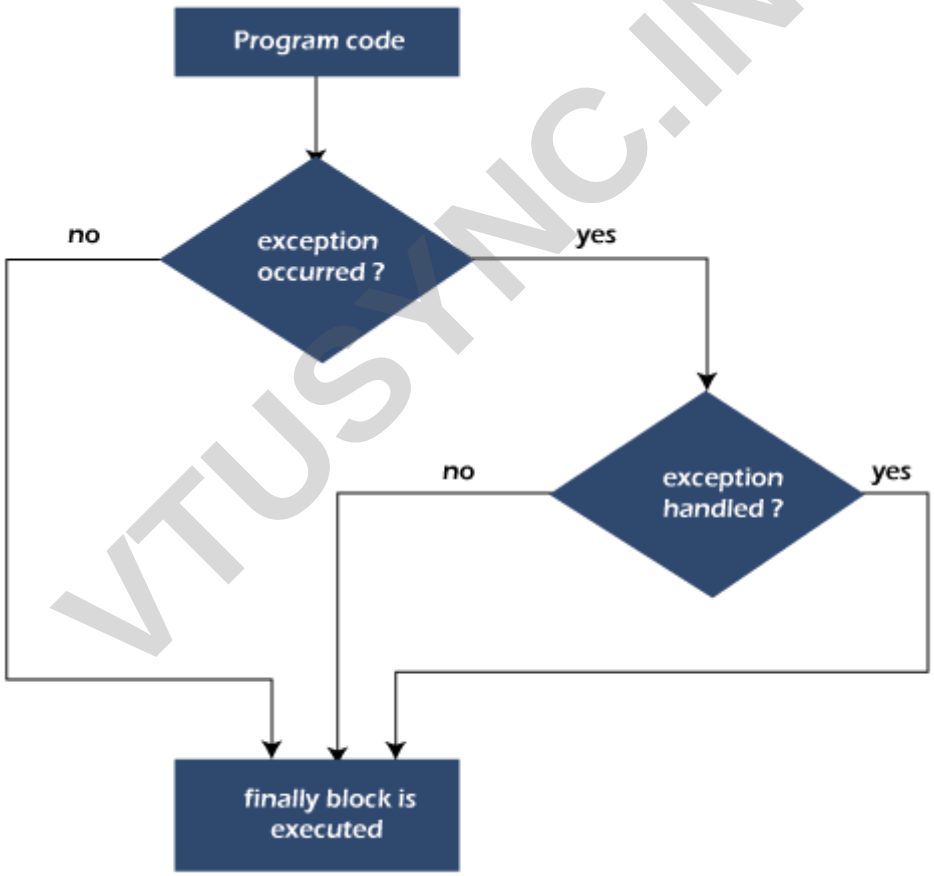
Java finally block

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

The finally block follows the try-catch block.

Flowchart of finally block



Note: If you don't handle the exception, before terminating the program, JVM executes finally block (if any).

Why use Java finally block?

- finally block in Java can be used to put "**cleanup**" code such as closing a file, closing connection, etc.
- The important statements to be printed can be placed in the finally block.

Usage of Java finally

Let's see the different cases where Java finally block can be used.

Case 1: When an exception does not occur

Let's see the below example where the Java program does not throw any exception, and the finally block is executed after the try block.

TestFinallyBlock.java

```
class TestFinallyBlock

 {

  public static void main(String args[])

{

  Try

{

//below code do not throw any exception

   int data=25/5;

   System.out.println(data);

  }

//catch won't be executed

  catch(NullPointerException e)

{

System.out.println(e);

}

//executed regardless of exception occurred or not

 finally

{
```

```
System.out.println("finally block is always executed");
}
System.out.println("rest of the code...");
 }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock.java

C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock
5
finally block is always executed
rest of the code...
```

Case 2: When an exception occurs but not handled by the catch block

Let's see the following example. Here, the code throws an exception however the catch block cannot handle it. Despite this, the finally block is executed after the try block and then the program terminates abnormally.

TestFinallyBlock1.java

```
public class TestFinallyBlock1
{
    public static void main(String args[])
   {
     try
    {
        System.out.println("Inside the try block");
       //below code throws divide by zero exception
     int data=25/0;
     System.out.println(data);
    }
    //cannot handle Arithmetic type exception
    //can only accept Null Pointer type exception
    catch(NullPointerException e)
    {
      System.out.println(e);
```

```
    }
   //executes regardless of exception occured or not
    finally
    {
      System.out.println("finally block is always executed");
    }
  System.out.println("rest of the code...");
    }
  }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock1.java

C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock1
Inside the try block
finally block is always executed
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at TestFinallyBlock1.main(TestFinallyBlock1.java:9)
```


Case 3: When an exception occurs and is handled by the catch block

Example:

Let's see the following example where the Java code throws an exception and the catch block handles the exception. Later the finally block is executed after the try-catch block. Further, the rest of the code is also executed normally.

TestFinallyBlock2.java

```
public class TestFinallyBlock2
{
    public static void main(String args[])
    {
    try
    {
```

```
    System.out.println("Inside try block");
    //below code throws divide by zero exception
 int data=25/0;
 System.out.println(data);
}
  //handles the Arithmetic Exception / Divide by zero exception
catch(ArithmeticException e)
 {
  System.out.println("Exception handled");
  System.out.println(e);
}

//executes regardless of exception occured or not
finally
{
  System.out.println("finally block is always executed");
 }
System.out.println("rest of the code...");
 }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestFinallyBlock2.java

C:\Users\Anurati\Desktop\abcDemo>java TestFinallyBlock2
Inside try block
Exception handled
java.lang.ArithmeticException: / by zero
finally block is always executed
rest of the code...
```

Rule: For each try block there can be zero or more catch blocks, but only one finally block.

Note: The finally block will not be executed if the program exits (either by calling System.exit() or by causing a fatal error that causes the process to abort).

Java throw Exception

In Java, exceptions allows us to write good quality codes where the errors are checked at the compile time instead of runtime and we can create custom exceptions making the code recovery and debugging easier.

Java throw keyword

The Java throw keyword is used to throw an exception explicitly.

We specify the **exception** object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.

We can **throw either checked or unchecked exceptions in Java by throw** keyword. It is mainly used to throw a custom exception. We will discuss custom exceptions later in this section.

We can also define our own set of conditions and throw an exception explicitly using throw keyword. For example, we can throw ArithmeticException if we divide a number by another number. Here, we just need to set the condition and throw exception using throw keyword.

The syntax of the Java throw keyword is given below.

throw Instance i.e.,

1. throw new exception_class("error message");

Let's see the example of throw IOException.

1. throw new IOException("sorry device error");

Where the Instance must be of type Throwable or subclass of Throwable. For example, Exception is the sub class of Throwable and the user-defined exceptions usually extend the Exception class.

Java throw keyword Example

Example 1: Throwing Unchecked Exception

In this example, we have created a method named validate() that accepts an integer as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

TestThrow1.java

In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
public class TestThrow1
{     //function to check if person is eligible to vote or not
  public static void validate(int age)
  {
```

```
        if(age<18)
         {          //throw Arithmetic exception if not eligible to vote
           throw new ArithmeticException("Person is not eligible to vote");
         }
        else
        {
           System.out.println("Person is eligible to vote!!");
         }
    }
    //main method
    public static void main(String args[])
    {
       //calling the function
       validate(13);
       System.out.println("rest of the code...");
     }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow1.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow1
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to
 vote
        at TestThrow1.validate(TestThrow1.java:8)
        at TestThrow1.main(TestThrow1.java:18)
```

The above code throw an unchecked exception. Similarly, we can also throw unchecked and user defined exceptions.

Note: If we throw unchecked exception from a method, it is must to handle the exception or declare in throws clause.

If we throw a checked exception using throw keyword, it is must to handle the exception using catch block or the method must declare it using throws declaration.

Example 2: Throwing Checked Exception

Note: Every subclass of Error and RuntimeException is an unchecked exception in Java. A checked exception is everything else under the Throwable class.

TestThrow2.java

```
import java.io.*;
```

```
public class TestThrow2
 {

   //function to check if person is eligible to vote or not
   public static void method() throws FileNotFoundException
{

      FileReader file = new FileReader("C:\\Users\\Anurati\\Desktop\\abc.txt");
      BufferedReader fileInput = new BufferedReader(file);



      throw new FileNotFoundException();

   }
   //main method
   public static void main(String args[])
   {
      try
      {
         method();
      }
      catch (FileNotFoundException e)
      {
         e.printStackTrace();
      }
      System.out.println("rest of the code...");
 }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow2.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow2
java.io.FileNotFoundException
        at TestThrow2.method(TestThrow2.java:12)
        at TestThrow2.main(TestThrow2.java:22)
rest of the code...
```

Example 3: Throwing User-defined Exception
exception is everything else under the Throwable class.

TestThrow3.java

```
// class represents user-defined exception
class UserDefinedException extends Exception
{
    public UserDefinedException(String str)
    {
        // Calling constructor of parent Exception
        super(str);
    }
}
// Class that uses above MyException
public class TestThrow3
{
    public static void main(String args[])
    {
        try
        {
            // throw an object of user defined exception
            throw new UserDefinedException("This is user-defined exception");
        }
        catch (UserDefinedException ude)
        {
            System.out.println("Caught the exception");
            // Print the message from MyException object
```

```
            System.out.println(ude.getMessage());
        }
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow3.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow3
Caught the exception
This is user-defined exception
```


Java throws keyword

The **Java throws keyword** is used to declare an exception. It gives an information to the programmer that there may occur an exception. So, it is better for the programmer to provide the exception handling code so that the normal flow of the program can be maintained.

Exception Handling is mainly used to handle the checked exceptions. If there occurs any unchecked exception such as NullPointerException, it is programmers' fault that he is not checking the code before it being used.

Syntax of Java throws

```
return_type method_name() throws exception_class_name
{
//method code
}
```

Which exception should be declared?

Ans: Checked exception only, because:

- **unchecked exception:** under our control so we can correct our code.
- **error:** beyond our control. For example, we are unable to do anything if there occurs VirtualMachineError or StackOverflowError.

Advantage of Java throws keyword

Now Checked Exception can be propagated (forwarded in call stack).

It provides information to the caller of the method about the exception.

Java throws Example

Let's see the example of Java throws clause which describes that checked exceptions can be propagated by throws keyword.

Testthrows1.java

```
import java.io.IOException;
class Testthrows1
{
 void m()throws IOException
 {
  throw new IOException("device error");//checked exception
 }
 void n()throws IOException
 {
  m();
 }
 void p()
 {
  try
  {
   n();
  }
   catch(Exception e)
   {
    System.out.println("exception handled");
```

```
        }
    }
        public static void main(String args[])
        {
         Testthrows1 obj=new Testthrows1();
         obj.p();
         System.out.println("normal flow...");
        }
}
```

Output:

```
exception handled
normal flow...
```

Rule: If we are calling a method that declares an exception, we must either caught or declare the exception.

There are two cases:

1. **Case 1:** We have caught the exception i.e. we have handled the exception using try/catch block.
2. **Case 2:** We have declared the exception i.e. specified throws keyword with the method.

Case 1: Handle Exception Using try-catch block

In case we handle the exception, the code will be executed fine whether exception occurs during the program or not.

Testthrows2.java

```
import java.io.*;
class M
{
        void method()throws IOException
       {
         throw new IOException("device error");
        }
```

```
}
public class Testthrows2
{
  public static void main(String args[])
  {
        try
       {
        M m=new M();
        m.method();
       }
      catch(Exception e)
      {
        System.out.println("exception handled");
      }

   System.out.println("normal flow...");
  }
}
```

Output:

```
exception handled
    normal flow...
```

Case 2: Declare Exception

- In case we declare the exception, if exception does not occur, the code will be executed fine.
- In case we declare the exception and the exception occurs, it will be thrown at runtime because **throws** does not handle the exception.

Let's see examples for both the scenario.

A) If exception does not occur

Testthrows3.java

```
import java.io.*;
```

```
class M
{
      void method()throws IOException
     {
      System.out.println("device operation performed");
      }
}
class Testthrows3
{
  public static void main(String args[])throws IOException   //declare exception
   {
    M m=new M();
    m.method();

   System.out.println("normal flow...");
}    }
```

Output:

```
device operation performed
    normal flow...
```

B) If exception occurs

Testthrows4.java

```
import java.io.*;
class M
{
 void method()throws IOException
       {
        throw new IOException("device error");
       }
}
class Testthrows4
{
  public static void main(String args[])throws IOException  //declare exception
  {
```

```
        M m=new M();
        m.method();
         System.out.println("normal flow...");
    }  }
```

Output:

```
Exception in thread "main" java.io.IOException: device error
    at M.method(Testthrows4.java:4)
    at Testthrows4.main(Testthrows4.java:10)
```

Difference between throw and throws

The throw and throws is the concept of exception handling where the throw keyword throw the exception explicitly from a method or a block of code whereas the throws keyword is used in signature of the method.

There are many differences between throw and throws keywords. A list of differences between throw and throws are given below:

Sr. no.	Basis of Differences	throw	throws
1.	Definition	Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
2.	Type of exception	Using throw keyword, we can only propagate unchecked exception i.e., the checked exception cannot be propagated using throw only.	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used to propagate checked exceptions only.
3.	Syntax	The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
4.	Declaration	throw is used within the method.	throws is used with the method signature.
5.	Internal implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by

			the method. For example, main() throws IOException, SQLException.

Difference between final, finally and finalize

The final, finally, and finalize are keywords in Java that are used in exception handling. Each of these keywords has a different functionality. The basic difference between final, finally and finalize is that the **final** is an access modifier, **finally** is the block in Exception Handling and **finalize** is the method of object class.

Along with this, there are many differences between final, finally and finalize. A list of differences between final, finally and finalize are given below:

Sr. no.	Key	final	finally	finalize
1.	Definition	final is the keyword and access modifier which is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java which is used to perform clean up processing just before object is garbage collected.
2.	Applicable to	Final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
3.	Functionality	(1) Once declared, final variable becomes constant and cannot be modified. (2) final method cannot be overridden by sub class. (3) final class cannot be inherited.	(1) finally block runs the important code even if exception occurs or not. (2) finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
4.	Execution	Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. It's execution is not dependant on the	finalize method is executed just before the object is destroyed.

			exception.	

Java final Example

Let's consider the following example where we declare final variable age. Once declared it cannot be modified.

FinalExampleTest.java

```
public class FinalExampleTest
{
  //declaring final variable
  final int age = 18;
  void display()
{
    // reassigning value to age variable
  // gives compile time error
  age = 55;
  }
  public static void main(String[] args)
{
 FinalExampleTest obj = new FinalExampleTest();
  // gives compile time error
  obj.display();
  }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac FinalExampleTest.java
FinalExampleTest.java:10: error: cannot assign a value to final variable age
        age = 55;
        ^
1 error
```

In the above example, we have declared a variable final. Similarly, we can declare the methods and classes final using the final keyword.

Java finally Example

Let's see the below example where the Java code throws an exception and the catch block handles that exception. Later the finally block is executed after the try-catch block. Further, the rest of the code is also executed normally.

FinallyExample.java

```
public class FinallyExample
{
    public static void main(String args[])
    {
            try
            {
             System.out.println("Inside try block");
            // below code throws divide by zero exception
             int data=25/0;
             System.out.println(data);
            }
    // handles the Arithmetic Exception / Divide by zero exception
             catch (ArithmeticException e)
            {
```

```
            System.out.println("Exception handled");
            System.out.println(e);
          }
    // executes regardless of exception occurred or not
        finally
        {
          System.out.println("finally block is always executed");
        }
    System.out.println("rest of the code...");
    }
  }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>java FinallyExample.java
Inside try block
Exception handled
java.lang.ArithmeticException: / by zero
finally block is always executed
rest of the code...
```


Java finalize Example

FinalizeExample.java

```
public class FinalizeExample
{
    public static void main(String[] args)
    {
```

```
        FinalizeExample obj = new FinalizeExample();
        // printing the hashcode
        System.out.println("Hashcode is: " + obj.hashCode());
        obj = null;
        // calling the garbage collector using gc()
        System.gc();
        System.out.println("End of the garbage collection");
    }
   // defining the finalize method
    protected void finalize()
    {
        System.out.println("Called the finalize() method");
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac FinalizeExample.java
Note: FinalizeExample.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\Anurati\Desktop\abcDemo>java FinalizeExample
Hashcode is: 746292446
End of the garbage collection
Called the finalize() method
```


Java Custom Exception

In Java, we can create our own exceptions that are derived classes of the Exception class. Creating our own Exception is known as custom exception or user-defined exception. Basically, Java custom exceptions are used to customize the exception according to user need.

Consider the example 1 in which InvalidAgeException class extends the Exception class.

Using the custom exception, we can have your own exception and message. Here, we have passed a string to the constructor of superclass i.e. Exception class that can be obtained using getMessage() method on the object we have created.

In this section, we will learn how custom exceptions are implemented and used in Java programs.

Why use custom exceptions?

Java exceptions cover almost all the general type of exceptions that may occur in the programming. However, we sometimes need to create custom exceptions.

Following are few of the reasons to use custom exceptions:

- To catch and provide specific treatment to a subset of existing Java exceptions.
- Business logic exceptions: These are the exceptions related to business logic and workflow. It is useful for the application users or the developers to understand the exact problem.

In order to create custom exception, we need to extend Exception class that belongs to java.lang package.

Consider the following example, where we create a custom exception named WrongFileNameException:

```
public class WrongFileNameException extends Exception

 {

   public WrongFileNameException(String errorMessage)

   {

   super(errorMessage);

    }

}
```

Note: We need to write the constructor that takes the String as the error message and it is called parent class constructor.

Example 1:

Let's see a simple example of Java custom exception. In the following code, constructor of InvalidAgeException takes a string as an argument. This string is passed to constructor of parent class Exception using the super() method. Also the constructor of Exception class can be called without using a parameter and calling super() method is not mandatory.

TestCustomException1.java

```
// class representing custom exception

class InvalidAgeException  extends Exception

{
```

```
    public InvalidAgeException (String str)
    {
        // calling the constructor of parent Exception
        super(str);
    }
}

// class that uses custom exception InvalidAgeException
public class TestCustomException1
{
     // method to check the age
    static void validate (int age) throws InvalidAgeException
     {
      if(age < 18)
    {

        // throw an object of user defined exception
        throw new InvalidAgeException("age is not valid to vote");
    }
      else
     {
      System.out.println("welcome to vote");
      }
     }

    // main method
    public static void main(String args[])
    {
        try
        {
            // calling the method
```

```
            validate(13);
        }
        catch (InvalidAgeException ex)
        {
            System.out.println("Caught the exception");

            // printing the message from InvalidAgeException object
            System.out.println("Exception occured: " + ex);
        }

        System.out.println("rest of the code...");
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestCustomException1.java

C:\Users\Anurati\Desktop\abcDemo>java TestCustomException1
Caught the exception
Exception occured: InvalidAgeException: age is not valid to vote
rest of the code...
```

Example 2:

TestCustomException2.java

```
// class representing custom exception
    class MyCustomException extends Exception
    {


    }
       // class that uses custom exception MyCustomException
    public class TestCustomException2
    {     // main method
       public static void main(String args[])
       {
          try
```

```
        {
            // throw an object of user defined exception
            throw new MyCustomException();
        }
        catch (MyCustomException ex)
        {
            System.out.println("Caught the exception");
            System.out.println(ex.getMessage());
        }
          System.out.println("rest of the code...");
    }
}
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestCustomException2.java

C:\Users\Anurati\Desktop\abcDemo>java TestCustomException2
Caught the exception
null
rest of the code...
```

Chained Exceptions :

Chained Exceptions in Java

In Java, a chained exception is a technique that enables programmers to associate one Exception with another. By providing additional information about a specific exception, debugging can be made easier. A chained exception is created by wrapping an existing exception in a new exception, which becomes the root cause of the new Exception.

The new Exception can provide additional information, while the original Exception contains the actual error message and stack trace. It makes it easier to determine and fix the problem's source. Chained exceptions are especially useful when an exception is thrown due to another exception.

In Java, a chained exception is created using one of the constructors of the exception class.

Throwable Class

Constructors and methods for supporting chained exceptions are available in the Throwable class. Let's start by taking a look at the constructors.

Throwable(Throwable cause): A Java constructor creates a new exception object with a specified cause exception.

Throwable(String desc, Throwable cause): A Java constructor creates a new Throwable object with a message and a cause. It allows for chaining exceptions and providing more detailed information about errors.

In Java, the following Throwable class methods enable chained exceptions:

getCause(): It is a Java method of the Throwable class that returns the cause of the current Exception. It allows for accessing the Exception or error that triggered the current Exception to be thrown.

initCause() method: determines the reason for the calling Exception.

Example of Chained Exception:

Filename: ChainedExceptionExample.java

```
1.  public class ChainedExceptionExample {
2.      public static void main(String[] args) {
3.          try {
4.              String s = null;
5.              int num = Integer.parseInt(s); // the line will throw a NumberFormatException
6.          } catch (NumberFormatException e) {
7.              // create a new RuntimeException with the message "Exception."
8.              RuntimeException ex = new RuntimeException("Exception");
9.
                // set the cause of the new Exception to a new NullPointerException with the message " It is actual ca
    use of the exception "
10.             ex.initCause(new NullPointerException("It is actual cause of the exception"));
11.             // throw the new Exception with the chained Exception
12.             throw ex;
13.         }
14.     }
15. }
```

Output:

```
Exception in thread "main" java.lang.RuntimeException: Exception at
ChainedExceptionExample.main(ChainedExceptionExample.java:7)
Caused by: Java.lang.NullPointerException: It is actual cause of the exception at ChainedExceptio
```